Simulator architecture: one choice, one run, one log line

COMMAND LINE (setup_argparse in main.py)

OPERATION plus --key_size, --data_size, --iterations, --debug, --show_time

main() DISPATCH: fresh random keys and data, one operation

- rsa_keygen
- rsa_encrypt
- rsa_decrypt
- aes_encrypt
- aes_decrypt
- hash, hmac, pbkdf2

simulate_timing_variation(base_time)

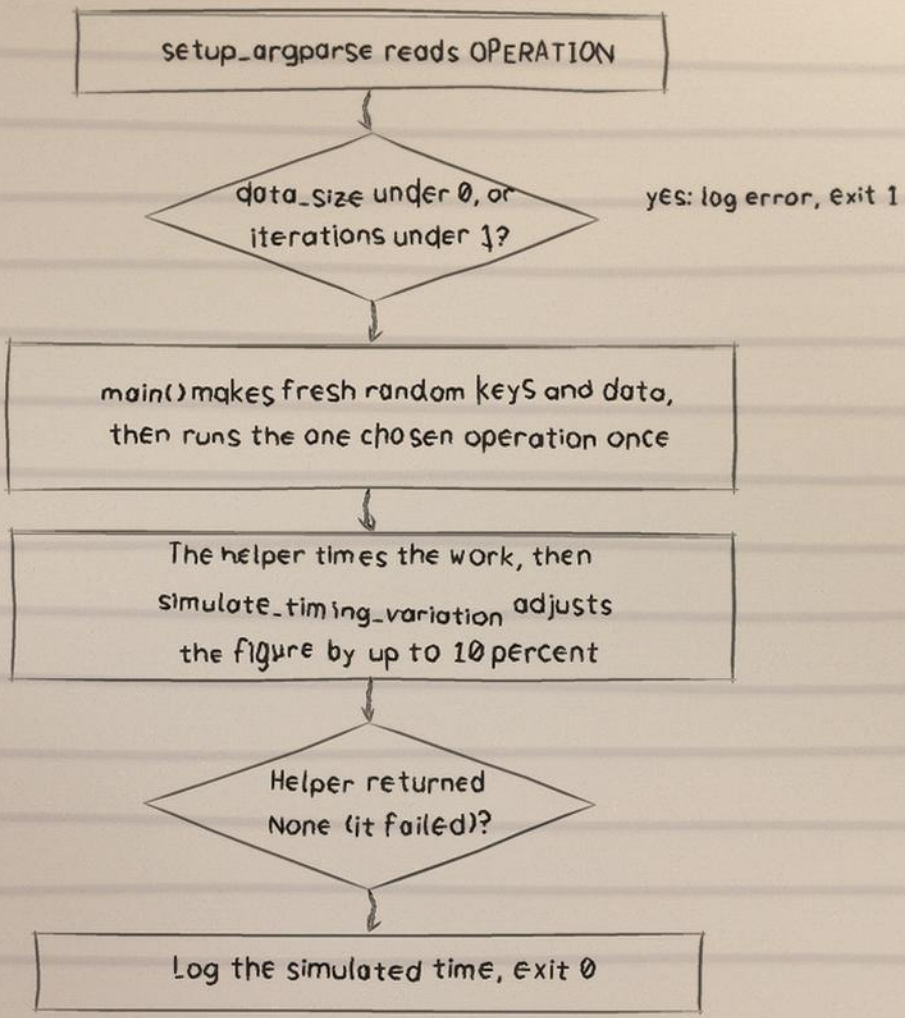
wall clock time, plus or minus up to 10 percent random change.
The logged figure is SIMULATED, not measured.

logging line with the simulated time, then exit 0 or exit 1

Keys, ciphertext, and digests are returned inside the program and never printed or saved.
One dependency: the cryptography package. RSA uses PKCS1v15 padding, AES uses CFB mode.

etched

One run, from flags to exit code



A failure logs its error and the run exits 1. --show_time only adds a note line.
For rsa_decrypt and aes_decrypt, the matching encrypt step runs first, inside the same run.

Product map: a timing demonstration, plainly labelled

WHAT IT IS

A small program that runs one real crypto operation on random data it makes itself, then logs a SIMULATED timing figure for that operation.

It is a demonstration of how a timing log reads. It is not an attack and it recovers no keys.

WHO IT SERVES

Me, learning how timing side channels are talked about, and anyone who wants the same first look.

INPUTS

One operation name and three numbers: key size, data size, iterations. No files are read.

OUTPUTS

Log lines only: one simulated time per run, plus error lines when an operation fails.

BOUNDARIES

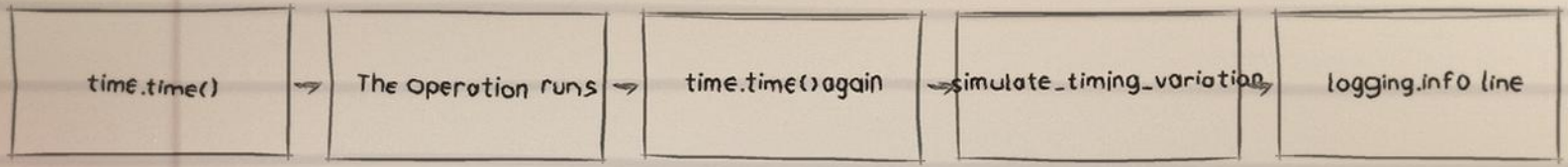
No power measurement. No key recovery. Keys and results are never printed or saved.

folder tab: GK

Operations: rsa_keygen, rsa_encrypt, rsa_decrypt, aes_encrypt, aes_decrypt, hash, hmac, pbkdf2.

Timing: how one simulated figure is made

Read this before quoting any number from this program: the logged time is simulated.

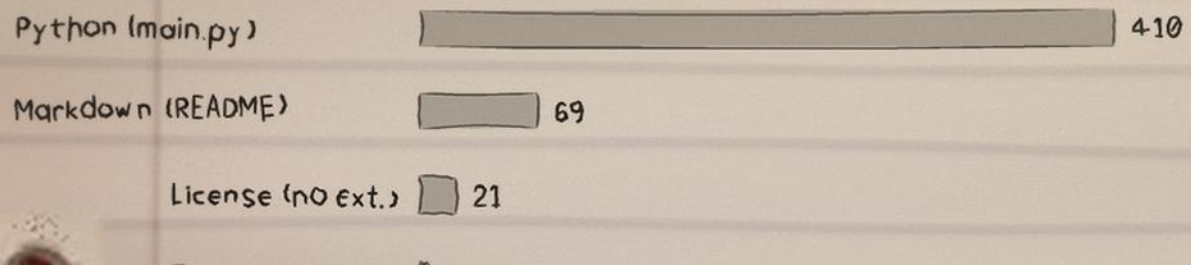


Every helper follows this same shape: rsa, aes, hash, hmac, and pbkdf2 alike.
A power trace is never taken. Nothing about a real chip is measured here.

timed on paper, OK

crypto-side-channel-simulator in numbers, counted 2026-10-30

Lines by kind, from git ls-files and a line count over each tracked text file.



The whole repo is 6 files and 8 commits. First commit 2025-05-28, last one 2026-10-30, both from git log. Tests: no test files and no test suite in this repo, so no pass is claimed. The cryptography package is not installed in my sandbox and installs are out of scope here, so the CLI itself was not run either.

counted and drawn OK

Crypto Side Channel Simulator: schematic of the apparatus

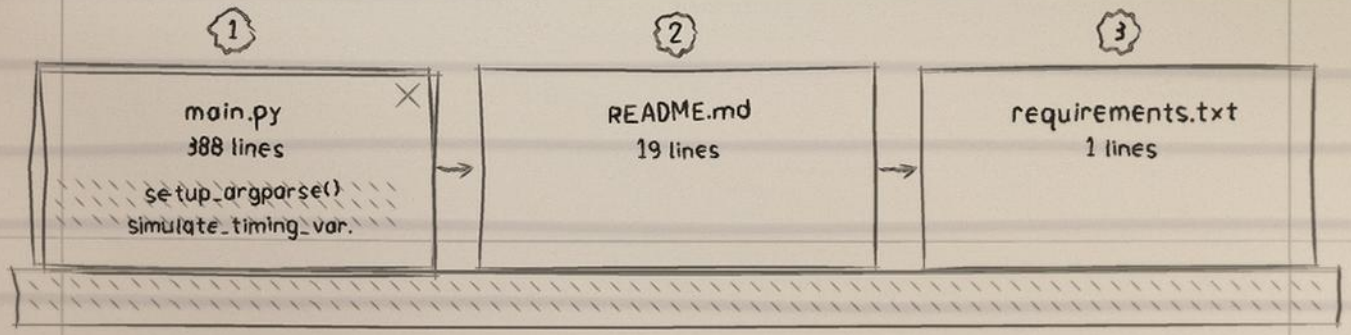
the system as a machine: parts, flow, and measurements

construction lines left in, as drawn

parts list

1. main.py
2. README.md
3. requirements.txt

in flow order, first to last



base plate: the repo itself, sectioned

span: 3 files, 408 lines counted

turns of the crank so far: 4 commits, 2025-05-28 to 2025-05-28

measured and sketched, GK